

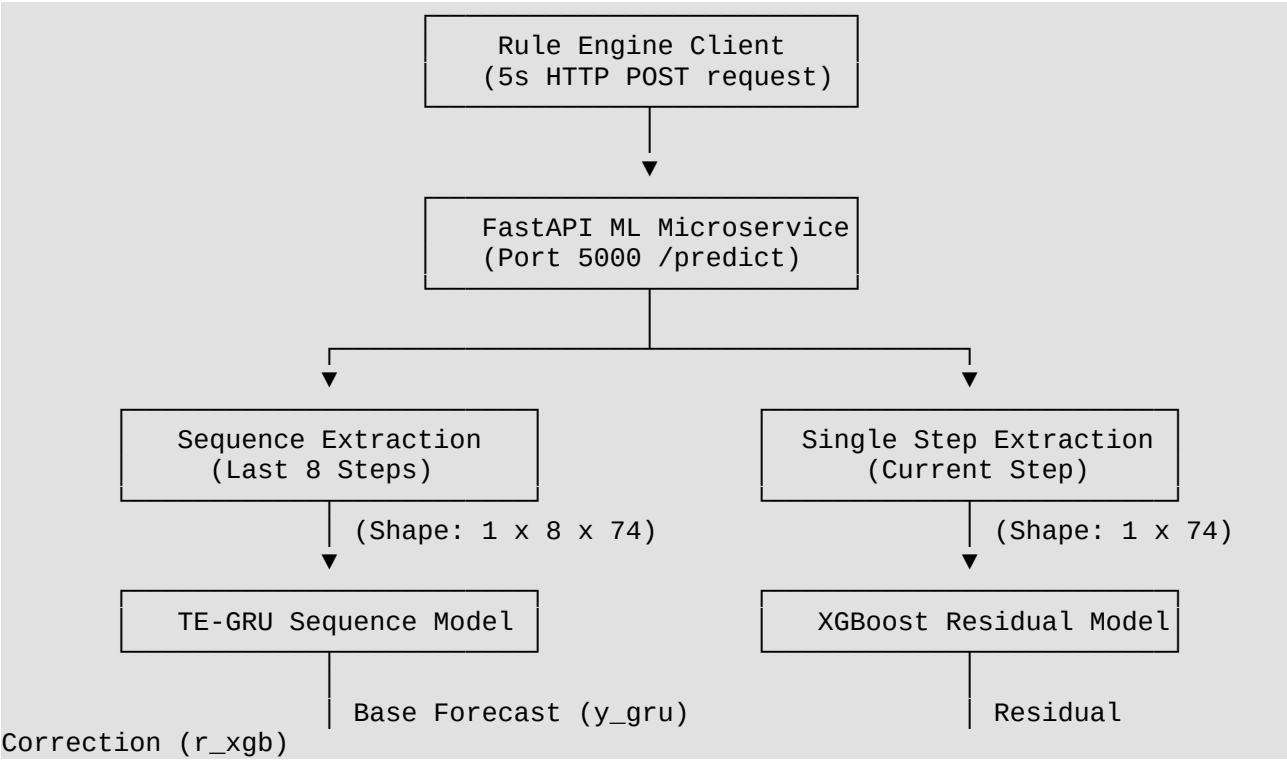
Model Integration Methodology: Smart Room Energy Management System

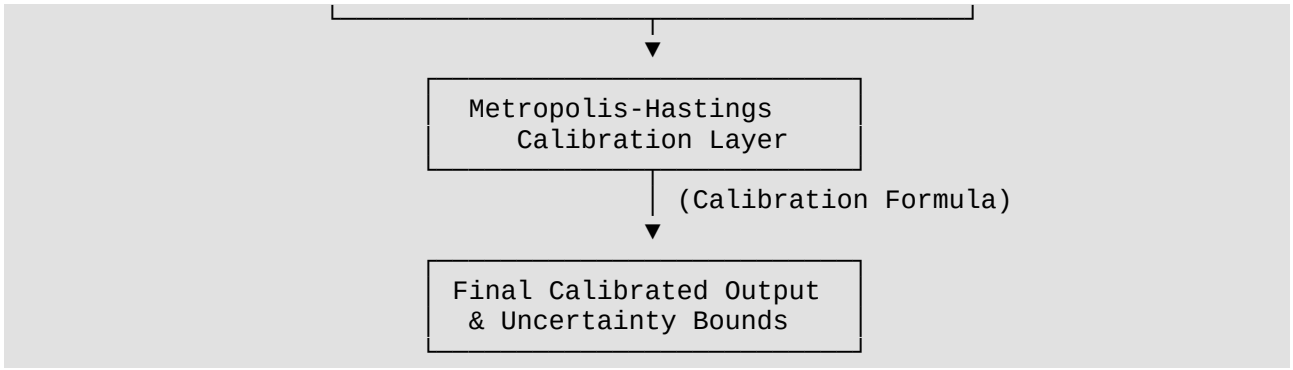
This document outlines the detailed system architecture, data flow, feature engineering pipeline, and integration mechanisms implemented to deploy the hybrid machine learning model within the Smart Room Energy Management System. This section is structured for direct inclusion or adaptation in the project methodology report.

1. System Architecture & Model Pipeline

The prediction subsystem employs a hybrid, multi-stage machine learning architecture designed to forecast short-term energy demand (measured in Watt-hours, Wh) over a 5-minute Energy Demand Forecast Interval (EDFI). The system integrates three distinct analytical components:

- Temporal Encoding Gated Recurrent Unit (TE-GRU):** A recurrent neural network (RNN) designed to process sequential data. It evaluates a temporal window of the last **8 consecutive steps** (representing 40 minutes of continuous history) to capture sequential dependency, baseline trends, and thermal/occupancy inertia.
- Extreme Gradient Boosting (XGBoost) Residual Regressor:** A tree-based ensemble model that operates on the current single timestep. Its primary function is to predict and correct the residual error of the TE-GRU's temporal forecast using the immediate room state.
- Metropolis-Hastings (MH) Calibration & Uncertainty Estimation:** A Bayesian blending layer that weights the predictions of the GRU and XGBoost models, applying a calibrated adjustment formula. It also calculates safety margins (80% and 95% prediction intervals) representing the statistical uncertainty of the forecast.





The final calibrated energy prediction ($\hat{y}$) is calculated using the outputs of the TE-GRU ($\hat{y}_{\text{GRU}}$) and XGBoost ($\hat{r}_{\text{XGB}}$) models according to the following equation:

$$\hat{y} = \hat{y}_{\text{GRU}} + \alpha \cdot \hat{r}_{\text{XGB}} + \beta$$

Where:

- $\alpha = 1.091811$ (Scaling parameter for the residual correction)
- $\beta = -0.085535$ (Additive intercept adjustment)
- $Q_{95} = 2.0577$ (95% confidence interval bound)
- $Q_{80} = 0.8464$ (80% confidence interval bound)

2. Feature Engineering Pipeline

The model does not operate on raw, isolated sensor readings. Instead, it utilizes an extensive feature engineering pipeline that transforms a raw 5-element telemetry vector into a **74-column feature matrix** representing the environmental, temporal, and historical state of the room.

For each of the 8 timesteps in the sequence window, the pipeline computes the following features:

2.1 Environmental Telemetry (4 Columns)

Direct measurements acquired from physical sensors:

- **temperature**: Room ambient temperature (DHT22 sensor, °C).
- **humidity**: Room relative humidity (DHT22 sensor, %).
- **lux**: Light intensity (BH1750 light sensor, lx).
- **occupancy**: Room occupancy count (estimated using ultrasonic and radar motion sensors).

2.2 Temporal & Calendar Indicators (6 Columns)

Time characteristics derived from the server-synchronized system clock:

- **hour**: Hour of the day (\$0\$ to \$23\$).

- **minute:** Minute of the hour (\$0\$ to \$59\$).
- **dayofweek:** Day of the week (\$0\$ to \$6\$, where \$0\$ is Monday).
- **month:** Month of the year (\$1\$ to \$12\$).
- **day:** Day of the month (\$1\$ to \$31\$).
- **is_weekend:** Binary flag (\$1\$ if the day is Saturday or Sunday; \$0\$ otherwise).

2.3 Trigonometric Time Encodings (4 Columns)

Trigonometric transformations applied to periodic temporal features to preserve cyclical continuity (e.g., ensuring 23:59 is mathematically adjacent to 00:01):

- **hour_sin / hour_cos:** Sine and Cosine representation of the hour:
$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot \text{hour}}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot \text{hour}}{24}\right)$$
- **month_sin / month_cos:** Sine and Cosine representation of the month:
$$\text{month_sin} = \sin\left(\frac{2\pi \cdot \text{month}}{12}\right), \quad \text{month_cos} = \cos\left(\frac{2\pi \cdot \text{month}}{12}\right)$$

2.4 Interaction Cross-Features (5 Columns)

Multiplicative combinations of live variables to capture non-linear relationships:

- **temp_x_humidity:** Temperature $\times$ Humidity
- **temp_x_occupancy:** Temperature $\times$ Occupancy
- **humidity_x_occupancy:** Humidity $\times$ Occupancy
- **lux_x_occupancy:** Light Level $\times$ Occupancy
- **hour_x_occupancy:** Hour $\times$ Occupancy

2.5 Historical Energy Lags & Statistics (22 Columns)

Temporal characteristics capturing past energy states. These are computed and duplicated under two separate namespace prefixes (**energy_** and **real time energy_**) to maintain compatibility with different model versions:

- **Lags:** lag1, lag2, lag3 (representing actual energy consumed 5, 10, and 15 minutes ago, respectively).
- **Rolling Averages:** roll3, roll6, roll12 (moving averages of energy consumption over the last 15, 30, and 60 minutes).
- **Rolling Volatility:** std6, std12 (moving standard deviation of energy over the last 30 and 60 minutes).
- **Trends:** trend3, trend6, trend12 (difference between the previous timestep and the energy level N steps prior).

2.6 Historical Sensor Lags & Statistics (33 Columns)

Lags, rolling averages, rolling standard deviations, and trends (matching the formulas above) are calculated for the environmental sensors:

- **Temperature history:** 11 columns (lags 1–3, rolling averages 3/6/12, rolling standard deviations 6/12, trends 3/6/12).
- **Humidity history:** 11 columns.
- **Occupancy history:** 11 columns.

Total Feature Count: $4 \text{ (raw)} + 6 \text{ (calendar)} + 4 \text{ (cyclical)} + 5 \text{ (interactions)} + 22 \text{ (energy lags)} + 33 \text{ (sensor lags)} = 74 \text{ columns}$.

3. Data Hydration & History Retrieval

Unlike typical stream-processing setups that maintain a long-running, in-memory queue inside the microservice, the FastAPI ML service implements a **stateless query model** that interfaces directly with the central database on a per-request basis.

3.1 SQLite History Extraction

When a prediction request is received at `/predict`, the microservice dynamically queries the SQLite database (`room_backend/db.sqlite3`), specifically reading the `energy_relaystate` table:

- **Query Parameters:** It fetches the last 60 records sorted chronologically.
- **Telemetry Extraction:** For each record, the service extracts the timestamp, temperature, humidity, lux, occupancy, and the 1-minute integrated energy (`energy_kw` in Wh).
- **Energy Normalization:** Since the database stores 1-minute integrated energy, the service scales the values by multiplying them by 5.0 to convert them into their 5-minute training block equivalents before passing them to the feature engineering pipeline.

3.2 Bootstrapping & CSV Fallback

To ensure robustness during cold starts (e.g., when the server first boots up and the database is empty or has fewer than 60 records):

- **CSV Padding:** If the database contains fewer than 60 records, the microservice accesses a local historical CSV file (`mydata_new.csv`) containing reference baseline telemetry.
 - **Backfilling:** The service extracts the necessary number of tail rows from the CSV file to pad the database history, ensuring a complete 60-row sequence is always available for lag and rolling calculations.
 - **Fallback Defaults:** If the CSV file is also missing or unreadable, the service populates the remaining rows with static default values (e.g., 25°C temperature, 50% humidity, 0 lux, 0 occupancy, and 11.6 Wh energy).
-

4. API & Integration Protocol

The ML service runs as a headless REST API utilizing **Uvicorn** and **FastAPI** on port 5000.

4.1 Prediction Request Schema

The rule engine issues HTTP POST requests to `/predict`. The body contains the immediate real-time environmental snapshot:

```
{
  "temperature_c": 30.5,
  "humidity": 61.2,
  "lux": 150.0,
  "occupancy": 3,
  "datetime_str": "2026-06-24T22:30:00"
}
```

Note: The current energy is omitted in the body, as that is the target variable. The historical energy lags for the prediction row are calculated using the previous step values retrieved from the SQLite database history.

4.2 Prediction Response Schema

The service returns a flat JSON payload containing the predictions, bounds, and contributing model values:

```
{
  "predicted_energy_wh": 13.4,
  "upper_bound_energy_wh": 15.4577,
  "lower_bound_energy_wh": 11.3423,
  "predicted_energy_range_wh": [11.3423, 15.4577],
  "energy_unit": "Wh",
  "tegru_raw_wh": 12.6476,
  "xgb_residual_wh": 0.7675,
  "peak_demand": 5.0,
  "buffer_size": 60,
  "timestamp": "2026-06-24T22:30:00.123456Z",
  "source": "fastapi-tegru-xgb-mh-db"
}
```

4.3 Control Loop vs. Dashboard Polling

A design challenge was coordinating the polling frequency:

- **Dashboard Polling (Real-Time):** The rule engine calls the `/predict` endpoint every **5 seconds** and publishes the result to `room/ml/predictions` on the MQTT broker. This ensures that the user interface has a high-fidelity, real-time prediction graph.
- **Control Decisions (5-Minute Boundary):** Physical relay actuation decisions are restricted to 5-minute intervals. The rule engine uses the latest prediction cached in memory when the 5-minute decision boundary is reached.

5. Decision Logic & Actuation

The predicted energy (referred to as the Energy Demand Forecast Interval, or EDFI) is fed directly

into the system's control logic to actuate the room's appliance relays.

5.1 The EDFI Threshold Rules

The system evaluates the room's energy mode based on the predicted energy load and the stability of the battery storage system:

Load Class	Condition	Battery $\geq 24.5\text{V}$	Battery $\geq 24.0\text{V}$	Battery $\geq 23.5\text{V}$	Battery $< 23.5\text{V}$
Peak	EDFI $\geq 25\text{ Wh}$	Mode A (All ON)	Mode B (HVAC+Frz)	Mode C (Frz only)	Mode C (Frz only)
Moderate	EDFI $\geq 10\text{ Wh}$	Mode B (HVAC+Frz)	Mode B (HVAC+Frz)	Mode C (Frz only)	Mode C (Frz only)
Baseline	EDFI $\geq 1\text{ Wh}$	Mode C (Frz only)	Mode C (Frz only)	Mode C (Frz only)	Mode C (Frz only)
Very Low	EDFI $< 1\text{ Wh}$	Mode C (Frz only)	Mode C (Frz only)	Mode C (Frz only)	Mode C (Frz only)

5.2 Battery Stability Filter

To prevent rapid relay switching (chattering) caused by transient voltage spikes or sags under load, the rule engine implements a temporal lag filter on the battery voltage:

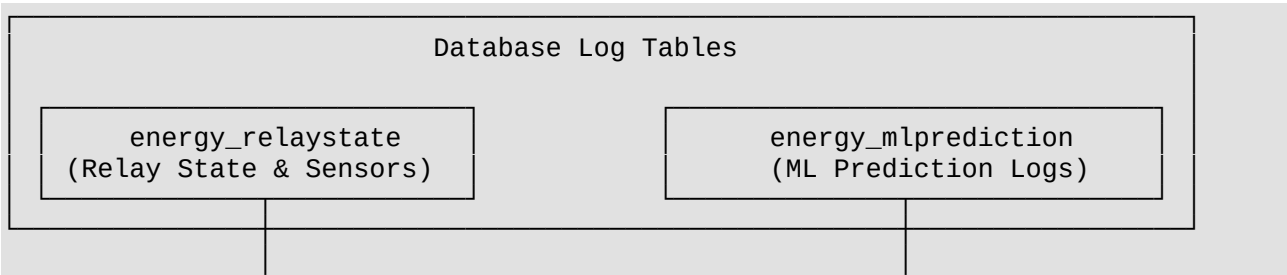
- **Tracking:** The system tracks the last 3 battery voltage readings sampled at 30-second intervals (ST_{now} , ST_{-1} , ST_{-2}).
- **Enforcement:** A mode can only be activated if all 3 historical readings in the stability queue are strictly above the threshold voltage.

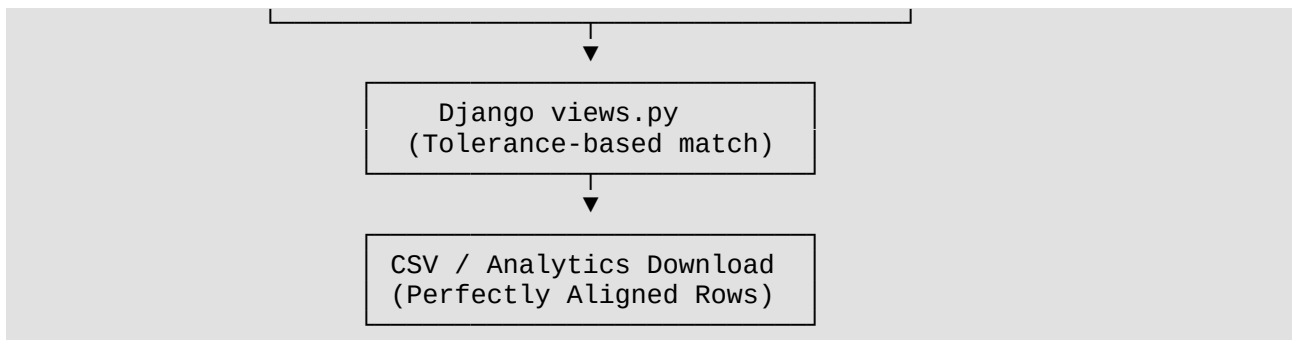
5.3 Actuation Pathway

The rule engine does not actuate the physical relays directly. Instead, it publishes the target states (relay_1, relay_2, relay_3) and the selected mode to the room/relays/state topic on the MQTT broker. An ESP32 microcontroller connected to the relay board subscribes to this topic and performs the physical actuation.

6. Database Logging & Alignment

All sensor readings, prediction outputs, and mode decisions are persisted to a SQLite database (db.sqlite3) managed by a Django backend.





6.1 Logging Desynchronization

- **Relay State Logging:** The Rule Engine evaluates decisions every 5 minutes and writes the room state immediately to the `energy_relaystate` database table.
- **Prediction Logging:** The MQTT Logger worker subscribes to `room/ml/predictions`, buffers the results, and flushes them to the `energy_mlprediction` table.
- **The Offset Challenge:** Because the prediction log is written asynchronously, the database timestamp for the prediction is offset by several seconds relative to the decision timestamp.

6.2 The Alignment Matching Algorithm

To align these records for post-hoc analysis and CSV downloads, the Django backend uses a tolerance-based future lookup window:

1. For each `RelayState` record at timestamp T_{decision} , the view queries the `MLPrediction` table.
2. It looks for the first prediction record with a timestamp $T_{\text{prediction}}$ that falls within a 6-minute window starting at the decision time: $T_{\text{decision}} - 30\text{s} \leq T_{\text{prediction}} \leq T_{\text{decision}} + 5\text{m } 30\text{s}$
3. This 30-second skew tolerance handles minor system clock offsets, and the 6-minute upper bound ensures that decisions are not matched with far-future predictions in the event of logging gaps.
4. This ensures that every row in the exported analytics output matches the exact environment state, the forecast generated for that state, and the resulting relay mode decision.